ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

FSP Talent

Платформа обратного подбора ИТ-специалистов по подтверждённым навыкам

Документ описывает назначение системы, архитектуру, установку, конфигурацию, пользовательские сценарии, API, безопасность, эксплуатацию и обновление проекта.

Версия документа: 1.0
Исходный код: ветка `agent/autonomous`, commit `aa3291f`
Дата: 11 октября 2026 г.

1. Назначение и возможности

FSP Talent — двухсторонняя платформа для подбора ИТ-специалистов. Кандидат создаёт профиль, указывает специализацию и заявленный грейд, проходит assessment и получает профиль доказательств. Работодатель создаёт hiring need, получает Talent Radar и может отправить приглашение с зарплатным диапазоном.

Основные возможности

- регистрация и подтверждение email через Keycloak;
- ограничение регистрации адресами домена `.ru`;
- отдельные кабинеты кандидата и работодателя;
- профиль, PDF-резюме и стандартизированный профиль;
- отрасль, специализация, заявленный грейд и системное тестирование;
- уникальные формы assessment с сопоставимой сложностью;
- Verified Skill Passport, FSP adapter и evidence freshness;
- Talent Radar, ProofMatch и blind discovery;
- вакансии, отклики, invitations, match и внутренний чат;
- hiring pipeline, задания работодателя и оценка результата;
- privacy/consent, minor consent, RBAC и серверная проверка JWT;
- OpenAPI, validation dashboard, Playwright QA и optional CPU-only local AI.

Роли

Роль	Назначение
Кандидат	Профиль, assessment, Skill Passport, вакансии, приглашения, match, чат и задания.
Работодатель	Профиль компании, hiring need, Talent Radar, invitations, вакансии, pipeline и employer tasks.
Администратор инфраструктуры	Docker, PostgreSQL, Keycloak, reverse proxy, TLS, резервное копирование и обновления.

2. Архитектура и структура проекта

Проект реализован как модульный монолит. Frontend, API, базы данных и identity-сервис запускаются в Docker Compose. Границы предметных областей сохраняются внутри backend, а внешние системы подключаются через adapters.

```
Browser
  │
  ▼
Next.js frontend ──────── Keycloak / OIDC
  │
  ▼
FastAPI REST + WebSocket
  ├── PostgreSQL application database
  ├── FSP Registry Adapter
  ├── ATS Connector Port
  ├── Resume Parser Adapter
  └── Optional Local AI Adapter

Mailpit — local SMTP sink
Playwright — browser QA
```

Путь	Содержимое
`backend/`	FastAPI, доменная логика, SQLAlchemy, Alembic и backend tests.
`frontend/`	Next.js, TypeScript, страницы и feature-компоненты.
`qa/`	Playwright-конфигурация, smoke/E2E и нагрузочный runner.
`infrastructure/keycloak/`	Шаблон realm и клиента `fsp-web`.
`scripts/`	Вспомогательные deployment, reset, resource и email-link scripts.
`docs/`	Архитектура, API, assessment, matching, безопасность и эксплуатационные материалы.
`docker-compose.yml`	Локальный Compose-контур.
`compose.prod.yml`	Production override: TLS reverse proxy network и закрытые host-порты.

3. Требования к окружению

- Docker Engine 24 или новее;
- Docker Compose v2;
- Git;
- для production — домен, TLS reverse proxy и внешняя Docker-сеть;
- для целевого CPU-only профиля — примерно 8 CPU и 32–64 GB RAM;
- GPU для основного контура не требуется.

Получение исходного кода

```
git clone git@github.com:Any1Key/fsp-platform.git
cd fsp-platform
git checkout main
```

Конфигурационные файлы

Файл	Назначение
`.env.example`	Шаблон переменных окружения без секретов.
`.env`	Локальные или deployment-specific значения; не коммитится.
`docker-compose.yml`	Базовые сервисы, volumes и healthchecks.
`compose.prod.yml`	Production-сеть reverse proxy и build-time frontend arguments.
`infrastructure/keycloak/realm-fsp.json`	Исходный шаблон Keycloak realm; существующий realm автоматически не обновляет.

4. Локальная установка

Создание .env

```
cp .env.example .env
```

Замените все значения `CHANGE_ME_*`. Минимальная локальная конфигурация:

```
APP_ENV=development
APP_BASE_URL=http://localhost:3000
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_KEYCLOAK_URL=http://localhost:8081
KEYCLOAK_PUBLIC_URL=http://localhost:8081
KEYCLOAK_ISSUER=http://localhost:8081/realms/fsp
CORS_ALLOWED_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
```

Запуск

```
docker compose config -q
docker compose up -d --build
docker compose ps
```

При старте backend применяет Alembic migrations. Frontend запускается после успешного healthcheck backend.

Адреса

Сервис	Адрес
Frontend	`http://localhost:3000`
Backend liveness	`http://localhost:8000/health`
Backend readiness	`http://localhost:8000/health/ready`
Swagger/OpenAPI	`http://localhost:8000/docs`
Keycloak	`http://localhost:8081`
Mailpit	`http://localhost:8025`

Остановка

```
docker compose down
```

Не используйте `docker compose down -v`, если нужно сохранить PostgreSQL, Keycloak и private-storage volumes.

5. Production-развёртывание

Публичные URL не зашиты в исходный код. Для каждого окружения используются свои значения в `.env`.

```
APP_ENV=production
APP_BASE_URL=https://your-domain.example
NEXT_PUBLIC_API_URL=https://your-domain.example
NEXT_PUBLIC_KEYCLOAK_URL=https://your-domain.example/auth
KEYCLOAK_PUBLIC_URL=https://your-domain.example/auth
KEYCLOAK_ISSUER=https://your-domain.example/auth/realms/fsp
KEYCLOAK_JWKS_URL=http://keycloak:8080/auth/realms/fsp/protocol/openid-connect/certs
CORS_ALLOWED_ORIGINS=https://your-domain.example
PROXY_NETWORK=your_reverse_proxy_network
```

Маршрутизация reverse proxy

- `/` → `frontend:3000`;
- `/api/*` → `backend:8000`;
- `/auth/*` → `keycloak:8080`;
- TLS завершается на reverse proxy;
- в Keycloak передаются `X-Forwarded-Proto: https` и `X-Forwarded-Host`.

Запуск production

```
docker compose -f docker-compose.yml -f compose.prod.yml config -q
touch email_confirmation_links.txt
docker compose -f docker-compose.yml -f compose.prod.yml up -d --build
docker compose -f docker-compose.yml -f compose.prod.yml ps
```

После изменения любого `NEXT_PUBLIC_*` значения frontend нужно пересобрать. Restart не изменяет значения, уже встроенные в Next.js bundle.

6. Keycloak, OIDC и email

Realm проекта — `fsp`, публичный клиент — `fsp-web`. Keycloak хранит пароли и выполняет регистрацию/подтверждение email. Приложение принимает только валидированный access token.

Настройка	Требование
Redirect URIs	`https://your-domain.example/*`
Web origins	`https://your-domain.example`
Flow	Authorization Code + PKCE; client public; Direct Access Grants disabled.

Issuer	В точности равен claim `iss` токена.
Audience	Содержит `fsp-web`.
Email	`email_verified=true`; допустим домен `.ru`.

Поток регистрации

1. Frontend открывает Keycloak по публичному HTTPS URL.
2. Keycloak создаёт пользователя и отправляет verification email через SMTP.
3. Пользователь переходит по ссылке.
4. Frontend получает токен через PKCE.
5. Backend проверяет RS256 signature, issuer, audience, exp/iat, sub, роль и verified email.
6. Пользователь без роли один раз проходит role onboarding.

Mailpit и файл ссылок

Локальный Compose использует Mailpit. Сервис `mailpit-link-writer` читает Mailpit API и записывает ссылки подтверждения в:

```
<корень проекта>/email_confirmation_links.txt
```

Файл исключён из Git. На новом сервере перед запуском создать его командой `touch email_confirmation_links.txt`. Посмотреть последнюю ссылку:

```
tail -n 1 email_confirmation_links.txt
```

> Ссылка подтверждения одноразовая и содержит токен. Ограничьте доступ к файлу и не публикуйте его.

Если PostgreSQL Keycloak уже существует, изменение realm JSON не обновляет существующий realm. Client редактируется через Admin Console или `kcadm` без удаления realm/database.

7. Backend, frontend и API

Backend

FastAPI предоставляет REST API под `/api/v1`, OpenAPI и WebSocket-операции для чата. Роутеры передают управление application services, доменные правила не зависят от FastAPI, persistence изолируется repositories.

Область	Функции
Identity	JWT, роли, onboarding и user identity.
Candidate	Профиль, resume, privacy, minor consent, FSP и Skill Passport.
Assessment	Создание, чтение и submit попытки; score и grade state.
Employer	Company profile, hiring need, Radar, invitations, vacancies и pipeline.
Matching	Category-first discovery и ProofMatch snapshots.

Chat	Создание conversation и сообщения только для участников match.
Tasks	Employer task, assignment, submission и evaluation.

Основные endpoints

Endpoint	Назначение
`GET /health`	Dependency-free liveness.
`GET /health/ready`	Readiness с проверкой PostgreSQL.
`POST /api/v1/auth/onboarding/role`	Однократная привязка роли.
`POST /api/v1/assessments/attempts`	Создание candidate-scoped assessment.
`POST /api/v1/assessments/attempts/{id}/submit`	Submit и deterministic scoring.
`GET /api/v1/candidate/skill-passport`	Self-scoped evidence projection.
`GET /api/v1/employer/talent-radar`	Talent Radar и ProofMatch.

Полный контракт доступен в Swagger: `/docs`, OpenAPI JSON — `/openapi.json`.

8. Assessment и grade states

Форма assessment создаётся по blueprint для пары specialization/grade. Blueprint определяет competency domains, веса, difficulty mix, critical floors, форматы и временной бюджет.

Защита от совпадения форм

- secure attempt-specific seed;
- семейства шаблонов с параметрическими вариантами;
- скрытые calibrated anchors;
- exposure budget и cooldown для недавних заданий;
- server-side timer и immutable form;
- answer keys и rubric secrets не выдаются клиенту;
- progressive follow-up и deterministic hidden checks;
- similarity/integrity signals без автоматического обвинения.

Статусы грейда

Статус	Смысл
`declared_unverified`	Грейд заявлен кандидатом, но не подтверждён assessment.
`verified`	Результат соответствует policy и critical floors.
`not_confirmed`	Попытка не подтвердила заявленный грейд; кандидат остаётся discoverable.
`eligible_for_lower_test`	Доступен тест нижнего грейда.
`eligible_for_higher_test`	Сильный результат позволяет предложить тест выше.

Политика по умолчанию: 80+ strong pass, 70–79 pass, 55–69 borderline, ниже 55 not confirmed. Retest cooldown по умолчанию 90 дней.

9. Matching, invitations и чат

Talent Radar начинает поиск с категории и hiring need, затем ранжирует кандидатов по покрытию обязательных и предпочтительных навыков, assessment, confidence, freshness, FSP и employer evidence.

ProofMatch

Для каждого результата возвращаются score breakdown, reasons, evidence sources, confidence, freshness и gaps. Кандидаты с неподтверждённым грейдом остаются видимыми с явной маркировкой.

Invitation и match

- salary min/max в рублях обязательны;
- кандидат видит компанию, offer и зарплату до общения;
- статусы: `sent`, `viewed`, `accepted`, `declined`, `expired`;
- `accepted` создаёт match;
- до match нет прямого чата и контактов;
- после match история и сообщения доступны только участникам.

Employer task

Задание привязано к match и оценивается работодателем. Его результат сохраняется как employer-specific evidence и не изменяет автоматически глобальный verified grade.

10. Данные, privacy и безопасность

Изоляция доступа

- backend получает subject из токена, а не из пользовательского параметра;
- candidate endpoints не принимают чужой `candidate_id` для self-scoped данных;
- employer видит только разрешённую evidence projection;
- conversation operations проверяют membership server-side;
- minor data и consent documents не участвуют в ranking.

Защита

- issuer/audience/signature/JWKS/exp/iat/sub/email_verified validation;
- RBAC, consent, privacy withdrawal/export/deletion concepts;
- CORS allowlist и security headers;
- rate limits и безопасное логирование;
- токены, пароли, полные сообщения, документы и лишняя PII не логируются;
- FSP identity отделена от Keycloak identity и соединяется через one-way subject fingerprint.

Production-обязательства

Для реальной эксплуатации нужны резервные копии PostgreSQL и Keycloak, production SMTP вместо Mailpit, управление секретами, мониторинг, retention/log access policy, ограничение Admin Console и регулярное обновление образов.

11. Миграции, данные и обновления

Миграции

```
docker compose exec backend alembic current
docker compose exec backend alembic upgrade head
```

Миграции применяются при запуске backend. Не удаляйте volumes при обычном обновлении.

Обновление

```
git fetch origin
git checkout main
git pull --ff-only origin main
cp .env .env.backup.$(date +%Y%m%d-%H%M%S)
docker compose -f docker-compose.yml -f compose.prod.yml config -q
docker compose -f docker-compose.yml -f compose.prod.yml up -d --build
docker compose -f docker-compose.yml -f compose.prod.yml ps
```

Демо-данные и reset

Скрипт reset является destructive-операцией и используется только для отдельного demo-окружения:

```
DEMO_RESET_CONFIRM=YES ./scripts/demo-reset.sh
```

> Reset удаляет volumes Compose-проекта. Он не используется на окружении с нужными данными.

Резервное копирование

```
docker compose exec -T app-db pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" > app-db-backup.sql
docker compose exec -T keycloak-db pg_dump -U "$KEYCLOAK_DB_USER" "$KEYCLOAK_DB" > keycloak-db-backup.sql
```

Файлы backup хранятся вне Git и шифруются в соответствии с политикой окружения.

12. Проверки и устранение проблем

Базовые health checks

```
docker compose ps
curl -fsS https://your-domain.example/health
curl -fsS https://your-domain.example/api/health/ready
curl -fsS https://your-domain.example/auth/realms/fsp/.well-known/openid-configuration
curl -fsS https://your-domain.example/api/docs
```

Targeted tests

```
docker compose exec -T backend pytest -q tests/test_invitations.py tests/test_chat.py tests/test_employer_tasks.py
docker compose exec -T backend pytest -q tests/test_assessments.py
docker compose --profile test run --build --rm qa \
 npx playwright test tests/smoke.spec.ts tests/security-headers.spec.ts tests/fsp-brand.spec.ts \
 --project=desktop-1920 --retries=0
```

Частые проблемы

Симптом	Проверка
OIDC session rejected	Сверить `iss` токена и `KEYCLOAK_ISSUER`, публичный URL, audience `fsp-web`, JWKS и redirect URI.
Старые URL во frontend	Изменить NEXT_PUBLIC_* и выполнить `up -d --build frontend`.
Нет письма	Проверить Mailpit, SMTP settings Keycloak, `docker compose logs keycloak` и файл links.
Readiness 503	Проверить app-db health, DATABASE_URL и логи backend.
Chat forbidden	Проверить, что invitation accepted и создан match; до match доступ закрыт намеренно.
Client settings did not change	JSON realm не обновляет существующий realm; править client через Admin Console/kcadm.

13. Эксплуатационные ограничения

- основной контур работает без GPU и без внешнего AI;
- local AI опционален, асинхронен и имеет deterministic fallback;
- FSP и ATS подключаются через adapters; синтетический FSP источник заменяется отдельной интеграцией;
- Mailpit предназначен для локальной/demo-среды, production использует SMTP-провайдера;
- нагрузочные показатели зависят от VPS, PostgreSQL и reverse proxy и проверяются на целевом окружении;
- полный authenticated E2E-сценарий должен выполняться отдельно после настройки конкретных аккаунтов и SMTP.

Связанные документы в репозитории

`ARCHITECTURE.md`, `DEPLOYMENT.md`, `API_GUIDELINES.md`, `ASSESSMENT_ENGINE.md`, `MATCHING_ENGINE.md`, `CHAT_AND_MATCHING.md`, `FSP_INTEGRATION.md`, `SECURITY.md`, `LEGAL_152FZ.md`, `REQUIREMENTS_MATRIX.md` и `DEMO_SCENARIO.md`.

Документ подготовлен по текущему исходному коду и документации проекта. Домены, секреты и deployment-specific параметры в документ не зашиваются.